Aritmética Takum na Extensão Vetorial RISC-V

Projeto Integrador AOC/UFMG

André Luís Alves Rezende Gabriel Ramos Fontes Isadora Maria Carvalho Lopes João Pedro Gonçalves de Noronha Dutra

Departamento de Engenharia Eletrônica – UFMG

2026

Motivação: proliferação de formatos

- RVV hoje: FP16 (`Zvfh`), bfloat16 (`Zvfbfmin`/`Zvfbfwma`), FP32/64, cripto. **FP8 não existe** na RVV.
- Cada formato novo adiciona conversões cruzadas, cantos de exceção e área de decodificador.
- A **aritmética Takum** propõe precisão afunilada com faixa dinâmica limitada e **um decodificador comum de 12 bits** para todas as larguras.
- Replicamos, na RVV, a metodologia do estudo de Hunhold para o AVX10.2 (MOCAST 2025).

Takum em 30 segundos

- Campos S, D, R, C, F; a característica cabe em 12 bits.
- T8/T16/T32: códigos curtos usam zeros à direita (janela de 12 bits).
- Negação = complemento de dois; NaR único; saturação; sem infinito.
- **Arredondamento RNE no código**: piso no prefixo de $n+1$ bits, arredonda pelo bit de meio-ponto e pelo *sticky*. Pode diferir do vizinho mais próximo em valor.
- Nesta versão: variante **linear** (Def. 8). A logarítmica fica para depois ($\Delta\pm$ tem custo próprio).

Zvtakum: cinco instruções, em custom-0

Mnemônico	Operação	Observação
`vtadd.vv`	$vd_i = vs2_i + vs1_i$	saturante
`vtsub.vv`	$vd_i = vs2_i - vs1_i$	saturante
`vtmul.vv`	$vd_i = vs2_i \times vs1_i$	saturante
`vtdot.vv`	$vd_0 \leftarrow vd_0 + \sum_i vs2_i \times vs1_i$	acúmulo 2n (T8→T16, ...)
`vncvt.t.t`	$T_{2n} \rightarrow T_n$	estreitamento RNE

- `opcode = 0x0B` (custom-0), `funct3 = 0`, `funct6` = operação, `vm` no bit 25.
- Por que não OP-V: o inventário do riscv-opcodes mostra que `funct6=0b111001` não é o único livre; custom-0 é o espaço reservado para extensões não padronizadas.
- SEW seleciona T8/T16/T32; máscara, cauda e `vl` da RVV; sem CSRs novos.

Arquitetura: núcleo RISC-V modelado

- **Escalar:** RV64IM + Zicsr, com `write` e `exit` (ABI Linux), então o kernel roda de verdade no simulador.
- **Vetorial (subconjunto RVV 1.0):** `vsetvl*`, `vle/vse` (8/16/32), `vluxei32`, `vmv`, `vfadd`, `vfmul`, `vfredusum`; VLEN configurável; LMUL 1–8.
- **Zvtakum:** unidade Takum com decodificador de 12 bits, aritmética exata em 128 bits, acumulador de $2n$ no `vtdot` e estreitamento no `vncvt`.
- **Contadores:** instruções por classe, escritas de CSR (inclui `vl`/`vtype` do `vsetvl`), elementos ativos e bytes.
- Fora do escopo: modo privilegiado, MMU, cache, FP8 e Zvfh.

Unidade Takum: aritmética própria, sem `double`

- Decodificação: janela de 12 bits $\rightarrow$ valor exato $(-1)^S \cdot \mathrm{sig} \cdot 2^{e-63}$.
- Soma/subtração: alinhamento do menor operando com bit *sticky*, normalização e codificação.
- Produto: significandos inteiros (até 28×28 bits em T32) e soma de expoentes, com 128 bits no acumulador T64.
- Codificação RNE no código, com saturação em maxpos/minpos.
- **Sem** `double` de propósito: as 128 divergências T32 registradas contra a libtakum vêm de arredondamento duplo no intermediário `double`.

Simulador e verificação

- `zt-sim` (C++17, três módulos): `takum.cpp`, `machine.cpp` e `main.cpp`. Carrega ELF64 e atende `write`/`exit`.
- **Núcleo Takum:** 21 verificações contra vetores dourados gerados pelo modelo Python (T8 completo, narrow T16→T8 completo, amostras de T16/T32/T64).
- **Diferencial com QEMU 10.2.2:** escalar e RVV padrão, byte a byte, em VLEN 128/256/512, incluindo máscara com elemento 0 inativo, `ta`/`ma`, `vl=0` e `vstart`.
- **Zvtakum contra o modelo Python:** 1 programa, 3 VLENs, byte a byte.
- **Kernels SpMV:** 96 execuções (8 matrizes $\times$ 4 formatos $\times$ 3 VLENs), todas bit a bit iguais ao modelo.

Metodologia experimental

- **Matrizes:** 8 da coleção Harwell–Boeing, de 30 a 1033 linhas e até 6027 não nulos.
- **Vetor x:** pseudoaleatório (semente 2026), elementos $k/2^{52}$: força arredondamento em FP32 e takum32.
- **Kernels:** mesmo laço escalar sobre o CSR; FP32 com `vfmul+vfredusum`; Takum com `vtdot+vncvt`; gather `vluxei32` nos dois.
- **Erro:** relativo L2 contra referência binary128 (113 bits, validada contra `__float128` do GCC).
- **Controles:** mesma matriz e mesmo x com acumuladores de larguras diferentes, para separar formato de acumulação.
- **Comparações pareadas** por matriz: contamos vitórias, derrotas e resultados não finitos.

Resultado 1: precisão em 32 bits

- Kernel FP32: erro mediano 5.91×10^{-8}; Takum32: 1.15×10^{-8}.
- Takum32 vence em 7 de 8 matrizes (exceção: `pores_1`).
- **Decomposição:** o ganho de 5.15 se divide em 2.01 do **formato**, com acumulador de 32 bits nos dois, e 2.56 do **acumulador** de 64 bits do `vtdot`.
- Com o mesmo acumulador de 32 bits, o Takum32 vence em 7 de 8 matrizes.
- Com acumulador de 64 bits nos dois, Takum32 é 3.15 vezes melhor.

Resultado 2: 8 e 16 bits

- Takum8 e Takum16 **não estouram** em nenhuma matriz. E4M3 estoura em 4, E5M2 em 3 e FP16 em 3 das 8.
- Onde nenhum estoura:
  - Takum16 vence FP16 (acumulador 32 bits) em 3 de 5;
  - Takum8 vence E5M2 (acumulador 16 bits) em 3 de 5;
  - E4M3 vence Takum8 em 4 de 4.
- Leitura honesta: o Takum troca precisão por ausência de overflow.

Resultado 3: custo de instruções

- Modelo analítico: $N = rR + cS + K$ (R = linhas, S = faixas), com parâmetros medidos e **resíduo zero** nas 96 execuções.
- Diferença do Takum32 para o FP32: $N_{\text{Takum32}} - N_{\text{FP32}} = R - S$. Um `vncvt` a mais por linha, um `vtdot` a menos por faixa.
- O Takum32 nunca executa mais que o FP32: 13 pares com menos, 11 iguais, 0 com mais. Escritas de CSR: idênticas.
- Em VLEN 256: FP32 4.59 instruções por não nulo, Takum32 4.55 (-0.7%).
- O de 8 e 16 bits paga um `vsetvli` extra por faixa (índice de 32 bits com LMUL maior): +27.4% e +29.9% mais escritas de CSR em VLEN 256.

Limitações e trabalhos futuros

- 8 matrizes e um único vetor x: ordens de grandeza, não intervalos de confiança.
- Estudo de 1401 matrizes (MuFoLAB) não executado.
- Variante logarítmica não implementada; FP16 vetorial (Zvfh) apenas reconhecida pelo montador.
- Contadores medem instruções, não ciclos, área ou energia.
- Futuro: 1401 matrizes, Zvfh, $\Delta\pm$, kernel sem a troca de configuração do índice e comparação com hardware.

Conclusão

- Extensão Zvtakum especificada e implementada em custom-0, com aritmética Takum própria em inteiros.
- Simulador funcional RV64IM+RVV+Zvtakum verificado contra o QEMU (byte a byte) e contra um modelo de racionais exatos.
- Takum32: erro menor que o FP32 em 7 de 8 matrizes com acumulador de mesma largura, e nunca mais instruções.
- Em 8 e 16 bits o custo depende do VLEN; o Takum não estoura, mas não é sempre mais preciso.

Obrigado! Perguntas?

Demo ao vivo

- `make sim-test` — núcleo Takum contra vetores dourados.
- `make sim-check` — diferencial com QEMU e com o modelo (9 execuções, byte a byte).
- Execução de um kernel no simulador, com contadores:
  `sim/build/zt-sim sim/build/spmv/bcsstk01.takum32.v256.elf --vlen 256 --stdout /tmp/y.bin --counters /tmp/c.json`
- `make spmv` — 96 kernels + FP8/FP16 + modelo de contagem (regenera os CSVs do artigo).